DreamNav — Hackathon + Research Project Specification v4.2

One-line summary

DreamNav turns a short validated phone walkthrough into a browser-navigable 3D Gaussian scene, assigns observed/predicted confidence using multi-view visibility support, and trains a geometrically consistent scene-specific completion model to predict nearby unseen viewpoints.

---

0. Executive verdict

DreamNav v4.2 is the sharpened build spec.

It keeps the hackathon-safe product clarity from v4, keeps the real ML backbone restored in v4.1, and adds the missing technical decisions needed to make the project credible for research conversations.

The project should not become a generic 3DGS viewer with AI branding. It should also not become an overloaded research system with too many fragile modules. The correct version is:

> **A reliable product demo built around a real, scene-specific, geometrically consistent ML model.**

The core system:

1. Upload or select a short validated phone walkthrough.
2. Recover camera poses using SLAM/SfM.
3. Build a 3D Gaussian Splatting reconstruction of the observed space.
4. Compute observed/partial/completion zones using multi-view visibility support.
5. Render pseudo-views from the reconstructed scene.
6. Train a lightweight scene-specific completion model.
7. Enforce geometric consistency between nearby predicted views.
8. Evaluate on held-out viewpoints.
9. Apply PSNR-based quality gates before showing completion as reliable.
10. Serve the scene in an interactive browser viewer with a confidence overlay.
11. Add production-design tools such as lens/FOV preview and camera markers.
12. Benchmark basic inference optimizations such as FP16, `torch.compile`, and optional ONNX/TensorRT.

This version satisfies both goals:

- **Hackathon goal:** polished, reliable, visually understandable demo.

- **Career/research goal:** real ML, world-model, 3D vision, and systems signal for RIPL, RL2, NVIDIA AMRI, embodied AI labs, ML systems roles, and research internships.

---

1. Project name

DreamNav

Recommended hackathon title:

> **DreamNav — AI Location Scout**

Recommended technical subtitle:

> **Geometrically consistent scene-specific world modeling from short phone walkthroughs**

Recommended resume subtitle:

> **Video-to-3D Gaussian reconstruction with visibility-aware neural scene completion**

---

2. Product thesis

Creative teams often scout locations through flat videos, photos, video calls, or expensive professional capture. But a director does not just want to watch a video. They want to walk through the space, test sightlines, preview lens choices, and decide whether the location is worth the budget.

DreamNav turns a short phone walkthrough into an interactive scouting scene.

A location scout records a 20–45 second walkthrough. DreamNav reconstructs the captured area into a navigable 3D Gaussian scene, computes what surfaces were actually supported by the camera views, then trains a lightweight model on that scene to predict nearby unseen viewpoints. The director opens the scene in a browser, walks through the captured space, toggles a confidence overlay to see what is real versus inferred, and previews camera/lens choices.

DreamNav is not a survey-grade scanner. It is a creative previsualization and spatial reasoning tool.

Product category

AI-assisted location scouting and previsualization.

Primary users

- film directors

- production designers
- location scouts
- commercial video teams
- indie filmmakers
- virtual production teams
- game environment artists

Secondary users

- architects
- interior designers
- real estate teams
- event planners
- creative agencies

Product wedge

DreamNav should not be pitched as a Matterport clone.

Better framing:

> **Matterport captures existing spaces accurately. DreamNav lets creative teams walk through a phone-captured space and understand what is observed versus AI-predicted.**

---

3. Technical thesis

DreamNav is a **scene-level world model for partially observed real environments**.

It is not a full embodied agent. It does not learn robot policies, manipulate objects, or output physical actions. But it does build the representation layer that embodied systems need:

- observed geometry
- camera trajectory
- novel view prediction
- partial observability
- confidence-aware scene completion
- queryable spatial memory
- geometric consistency across viewpoints

The key technical idea:

> **Use 3DGS for what was actually observed, define observation confidence from multi-view visibility, and use a geometrically consistent scene-specific neural model for nearby unseen or partially observed regions.**

This is what makes DreamNav more than a viewer.

4. What DreamNav is

DreamNav is an end-to-end system that takes a short validated phone video and produces an interactive browser scene.

It combines:

1. **Pose estimation**
2. SLAM/SfM recovers the camera trajectory.
3. **3D reconstruction**
4. 3D Gaussian Splatting reconstructs the observed region.
5. **Visibility-based confidence assignment**
6. Each Gaussian or spatial cell receives an observed/partial/predicted status based on how many input cameras directly supported it.
7. **Scene-specific learning**
8. A lightweight model trains on pseudo-views rendered from the scene.
9. **Geometric consistency**
10. Predictions from nearby viewpoints are constrained to agree after warping/projection.
11. **Novel-view completion**
12. The model predicts nearby partial/unseen views from target camera poses.
13. **Confidence-aware rendering**
14. The UI separates captured geometry from predicted regions.
15. **Product interaction**
16. Users navigate, preview lenses, mark camera positions, and inspect quality metrics.

5. What DreamNav is not

DreamNav is not:

- a VLA system
- a JEPA system in the hackathon build
- a robot-control project
- a manipulation policy
- a survey-grade scanner
- a full Matterport replacement
- a guaranteed arbitrary-video-to-perfect-3D system
- a pure generative-video system
- a physics simulator
- a complete embodied AI agent

Correct framing:

> **DreamNav is an embodied-AI-adjacent scene representation project, not a full embodied agent.**

---

6. Core contribution

Hackathon contribution

> **A browser-based AI location scouting tool that reconstructs a real space from phone video and makes observed versus model-predicted regions inspectable through a confidence overlay.**

ML/research contribution

> **A geometrically consistent scene-specific completion model trained from rendered views of a reconstructed 3D Gaussian scene to predict nearby unseen viewpoints under partial observability.**

3D vision contribution

> **A visibility-supported observed/completion boundary that assigns confidence based on whether surfaces were directly seen from multiple input cameras.**

Systems contribution

> **An interactive pipeline combining 3D reconstruction, lightweight scene adaptation, cached model outputs, basic inference optimization, and browser-based Gaussian rendering for smooth exploration.**

Product contribution

> **A focused creative workflow for remote scouting: walk the space, test lenses, mark camera positions, and understand uncertainty.**

---

7. Correct technical identity

Does it use 3DGS?

Yes. 3D Gaussian Splatting is the core observed-scene representation.

The observed region is rendered using 3DGS because it is:

- photorealistic
- fast enough for browser viewing
- suitable for real captured spaces
- visually impressive in a demo

Does it use SLAM?

Yes. The system needs camera poses.

DreamNav should use a two-tier strategy:

Hackathon reliability path

Use COLMAP/SfM or an existing 3DGS reconstruction pipeline for locked demo scenes if that is the most reliable path.

Online/research path

Use DROID-SLAM or equivalent visual SLAM as the intended real-time pose backend.

Recommended wording:

> “For the hackathon demo, DreamNav supports prevalidated scenes reconstructed with a reliable SLAM/SfM backend. The intended online path uses DROID-SLAM-style camera tracking for faster video-to-scene turnaround.”

This avoids making DROID-SLAM a fatal blocker while still preserving the SLAM/embodied AI signal.

Does it use ML?

Yes. ML is central.

DreamNav uses ML in:

- monocular/depth support if used
- feature extraction if used
- scene-specific completion
- geometric consistency learning
- novel-view prediction
- confidence estimation
- optional VLM scene manifest

The most important ML component is:

> **the geometrically consistent scene-specific completion model.**

Does it use VLA?

No. Do not claim VLA.

VLA means Vision-Language-Action. DreamNav does not output robot actions. It is a visual-spatial scene model.

Possible future VLA extension:

- use DreamNav scenes as environments for instruction-conditioned navigation agents
- add instruction-to-camera-path behavior
- add policy learning over predicted scene states

Not hackathon scope.

Does it use JEPA?

No, not in the hackathon path.

JEPA is a future research extension for latent predictive scene modeling and uncertainty estimation.

Possible future direction:

> Predict latent embeddings of unseen views instead of RGB pixels, then use latent prediction error as an uncertainty signal.

Not MVP.

---

8. World model / embodied AI framing

DreamNav should be framed as:

> **A scene-level visual world model for partially observed real environments.**

It has strong world-model signal because it supports:

- partial observability
- queryable scene memory
- novel-view prediction
- observation-backed versus predicted state separation
- uncertainty visualization
- scene-specific adaptation
- geometric consistency across nearby predicted views

It is embodied-AI-adjacent because it creates the representation infrastructure an embodied agent would need before planning or acting.

Good sentence for researchers

> "DreamNav is not a full embodied agent; it is a scene representation layer for embodied intelligence — a queryable visual-spatial world model that separates observation-backed geometry from geometrically constrained model-predicted regions."

Good sentence for hackathon judges

> "It lets you walk inside a phone video, and it shows you what was really captured versus what the AI had to infer."

---

9. Main demo narrative

Use the stronger concrete story from v3, but with honest v4.2 technical claims.

Opening story

> "A location scout in Detroit films a warehouse for 30 seconds. The director in LA does not just need another video — they need to walk the space, test sightlines, and decide whether this location is worth the budget. DreamNav turns that phone walkthrough into an interactive scouting scene."

Core demo sentence

> "The captured region is rendered as a Gaussian splat. The nearby predicted region comes from a scene-specific model trained on this space with a geometric consistency loss. The overlay shows exactly where the system switches from observed geometry to model prediction."

Technical close

> “Technically, DreamNav is a scene-specific world model for partial observations. It reconstructs what was seen, assigns confidence from multi-view visibility support, trains on rendered views of that scene, predicts nearby unseen viewpoints, and exposes uncertainty instead of hiding it.”

Product close

> “The point is not to replace professional scanning. The point is to let creative teams explore locations earlier, remotely, and with explicit confidence about what is real versus inferred.”

---

10. Critical-path feature list

Must-have

These define the hackathon MVP.

1. **Prevalidated demo scene**
2. at least one locked scene that works reliably
3. ideally 2–3 scenes by final demo
4. **Original input video display**
5. show the phone walkthrough before showing the result
6. **SLAM/SfM pose recovery**
7. produce camera trajectory
8. show trajectory on minimap
9. **3DGS reconstruction**
10. browser-loadable splat
11. first-person navigation
12. **Visibility-based observed/completion boundary**
13. assign observed/partial/completion status using multi-view camera support

14. must be implemented before the overlay is treated as real

15. **Scene-specific completion model**

16. train/adapt per scene

17. evaluated on held-out views

18. outputs predicted nearby views or completion regions

19. **Geometric consistency loss**

20. default training objective includes a geometric consistency term

21. not optional for the core research story

22. **PSNR quality gate**

23. prevent bad completion from being shown as reliable

24. use provisional thresholds and calibrate on demo scenes

25. **Observed versus predicted overlay**

26. the most important UI/technical feature

27. must be obvious to judges

28. **Minimap**

29. camera path

30. current user position
31. observed zone

32. predicted/completion zone

33. **Lens/FOV preview**

34. production-design relevance

35. 24mm / 35mm / 50mm / 85mm or simple equivalent

36. **Metrics panel**

37. splat FPS

38. model training time
39. held-out PSNR
40. completion latency or cache status
41. capture quality score

42. optimization mode used: FP32/FP16/compiled/TensorRT if available

43. **Basic inference optimization benchmark**

44. PyTorch eager FP32 baseline

45. FP16
46. `torch.compile` if available
47. optional ONNX/TensorRT

48. cached-output serving path

49. **Demo-safe cached model outputs**

50. cache scene-model predictions along the planned demo path

51. do not frame caching as the ML contribution
52. frame it as serving optimization

Should-have

1. Camera bookmarks
2. Quality report page
3. 2–3 demo scenes
4. Upload validation UI
5. Basic share link

Nice-to-have

1. Scene notes
2. VLM scene manifest
3. Exported virtual flythrough
4. Additional quality presets

Stretch only

1. Door interaction
2. Generated corridor
3. DROID-SLAM live online pipeline
4. Custom CUTLASS kernels
5. JEPA latent world model
6. VLA/navigation agent

---

11. Completion model — final decision

The completion module is no longer open-ended.

Default decision:

> **DreamNav trains a lightweight scene-specific pose-conditioned completion model on rendered pseudo-views from the reconstructed 3D Gaussian scene, with geometric consistency enforced across nearby predicted views.**

Fallbacks exist only for demo safety.

Why this decision

This gives DreamNav real ML signal.

It proves:

- scene-specific learning
- novel-view prediction
- partial observability handling
- geometric consistency
- measurable generalization
- world-model relevance

It is feasible if scoped correctly:

- low resolution
- small model
- short training
- limited completion region
- cached outputs for the live demo

What the model predicts

The model predicts a target view or completion patch from a target camera pose and scene context.

Minimum output:

- low-resolution RGB prediction for a nearby unseen/partial viewpoint

Better output:

- RGB + confidence mask

Best output if feasible:

- RGB + depth/geometry consistency signal + confidence mask

Input options

Minimum model input:

- target camera pose
- positional/pose encoding

Better input:

- target camera pose
- 2–4 nearest reference views
- relative pose between target and reference views

Best input if feasible:

- target camera pose
- Plücker ray or camera-ray embedding
- nearest reference-view image tokens
- observed/completion mask

Architecture options

Keep the model small.

Recommended architecture for hackathon:

1. **Pose-conditioned U-Net / image decoder**
2. easiest to reason about visually
3. **Tiny transformer encoder-decoder**
4. stronger research flavor
5. more risk
6. **MLP + convolutional decoder**
7. simple and fast

Recommended default:

> **Small pose-conditioned encoder-decoder trained at 256×256 or 384×384 resolution.**

Do not try to train a large NeRF-like model unless you already have it working.

---

12. Training objective

The training objective should not be plain RGB reconstruction only.

Plain photometric loss risks making the model a learned image interpolator. The research value comes from pushing predictions to be geometrically consistent across nearby viewpoints.

Default loss

```
L = L_rgb + λ_geo L_geometric_consistency
```

Starting value:

```
λ_geo = 0.1
```

Treat `λ_geo` as a hyperparameter. Log `L_rgb` and `L_geometric_consistency` separately. If one term dominates the other by an order of magnitude, adjust the weight.

Optional stronger loss

```
L = L_rgb + λ_geo L_geometric_consistency + λ_perc L_perceptual
```

Components

RGB loss

Use L1 by default.

```
L_rgb = || pred_rgb - target_rgb ||_1
```

L1 is preferred over pure L2 because L2 tends to produce blur.

Geometric consistency loss

Default depth source:

> **Use splat-rendered depth as the default depth source for geometric consistency.**

This is the right default because it is already produced during pseudo-view rendering, is grounded in the reconstructed 3DGS scene, and avoids adding a separate monocular depth model to the Week 3 critical path.

Fallback depth sources, only if splat-rendered depth is too noisy:

1. aligned monocular depth,
2. local homography for very small viewpoint shifts,
3. feature-space consistency.

For two nearby predicted views A and B:

1. Predict RGB for view A.
2. Predict RGB for nearby view B.
3. Use splat-rendered depth for view A.
4. Warp prediction A into view B using camera geometry.
5. Penalize disagreement in valid overlapping pixels.

Conceptually:

```
L_geo = || warp(pred_A, depth_A, pose_A -> pose_B) - pred_B ||_1 over valid
overlap
```

If depth is unavailable or unreliable, approximate geometric consistency using:

- splat-rendered depth
- monocular depth aligned to scene scale
- local homography for small viewpoint shifts
- feature-space consistency between nearby views

The first implementation can be approximate. The important thing is that the model is constrained across views, not trained as independent RGB frame prediction.

Perceptual loss

Optional.

Use only if the model already trains correctly.

Do not add perceptual loss before verifying:

- pose conditioning works
- model can overfit a tiny subset
- train/pose pairs are aligned

---

13. Observed/completion boundary definition

This is a core technical decision and must be implemented before Week 2 ends.

The overlay should not be based only on distance from the camera path. Distance is convenient but wrong: a point can be near the path and still be hidden behind a wall or object.

Final decision

> **Observed/completion status is based primarily on multi-view visibility support.**

Implementation strategy:

1. **Week 2 default:** start with a voxel/spatial-cell visibility approximation to get the overlay working quickly.
2. **Week 3 upgrade:** replace or refine it with per-Gaussian visibility once the overlay logic is stable.

The voxel version is acceptable for the hackathon demo if clearly represented as a visibility approximation. The per-Gaussian version is more precise and more defensible for research conversations.

A Gaussian or spatial cell is considered observed if it was directly visible from enough input cameras with sufficient geometric/photometric support.

Practical rule

For each Gaussian or spatial cell `g`:

1. Project `g` into each input camera.
2. Check whether the projected point is inside image bounds.
3. Check depth consistency against rendered/estimated depth.
4. Check opacity or photometric support if available.
5. Count valid supporting cameras.

Define:

```
visibility_count(g) = number of input cameras where g is visible and depth-
consistent
```

Zone assignment

Recommended provisional thresholds:

```
Observed:
    visibility_count >= 3
```

```
Partially observed:
    visibility_count in {1, 2}

Completion candidate:
    visibility_count == 0
    and g is near the observed boundary or target extrapolation region

Unknown/invalid:
    visibility_count == 0
    and g is too far from observed support
```

Overlay colors

Zone	Meaning	Display
Observed	directly supported by multiple input views	no tint
Partially observed	weakly supported / edge region	light teal
Completion	model-predicted region	teal/blue
Low confidence	failed quality gate or weak support	amber/red
Unknown/invalid	too far from reliable support	hidden or blocked

Why this matters

This lets you say:

> “The overlay is not arbitrary. It is computed from multi-view visibility support.”

That sentence is strong for both judges and researchers.

---

14. Completion pipeline

Step 1 — Reconstruct observed scene

Use SLAM/SfM + 3DGS to reconstruct what was seen.

Output:

- splat file
- camera poses
- sparse/dense scene metadata
- original camera trajectory

Step 2 — Compute visibility support

Before training the completion model, compute observed/partial/completion zones.

Output:

- visibility count per Gaussian or spatial cell
- observed mask
- partially observed mask
- completion candidate mask
- unknown/invalid mask

Step 3 — Render pseudo-views

Render training views from the 3DGS.

Generate:

- 300–800 train views if feasible
- 30–100 validation/held-out views if feasible
- lower count acceptable for hackathon if time constrained

Views should include:

- poses near the original camera trajectory
- small perturbations around observed camera poses
- slightly extrapolated poses near completion regions

Step 4 — Split train/validation

Hold out a subset of views.

Minimum:

- 80/20 split

Purpose:

- show the model can generalize to unseen nearby viewpoints
- produce meaningful metrics

Step 5 — Train scene-specific model

Train on the current scene only.

Default loss:

```
L = L_rgb + λ_geo L_geometric_consistency
```

Optional:

```
L = L_rgb + λ_geo L_geometric_consistency + λ_perc L_perceptual
```

Step 6 — Evaluate held-out views

Metrics:

- PSNR
- LPIPS if easy
- SSIM if easy
- qualitative side-by-side
- training time
- inference latency

Must show at least:

- training curve
- held-out PSNR
- held-out qualitative comparison

Step 7 — Apply quality gate

Use provisional gates:

```
median held-out PSNR >= 22 dB:
    completion demo enabled
    overlay toggle enabled

20 dB <= median held-out PSNR < 22 dB:
    completion shown only with warning / overlay on by default
    label as low confidence

median held-out PSNR < 20 dB:
    disable seamless completion demo
    fallback to observed-only or diagnostic overlay mode
```

These thresholds can be calibrated after testing real demo scenes.

Step 8 — Runtime rendering

Observed region:

- rendered directly by 3DGS

Completion region:

- rendered by scene-specific model or cached model output

Overlay:

- no tint for observed
- light teal for partially observed
- teal/blue for model-predicted
- amber/red for low confidence

Step 9 — Demo-safe caching

For the judged demo:

- precompute model predictions along the planned camera path
- store them as cached model outputs
- use them to guarantee smooth navigation

Important wording:

Do not say:

> “The completion is cached views.”

Say:

> “The scene-specific model generates the completion. For demo smoothness, predictions along the walkthrough path are cached.”

---

15. Reconstruction / pose backend decision

The spec should not leave the pose backend completely open.

Default plan

Use the most reliable backend available for the first working demo scene.

Recommended order:

1. Existing 3DGS/COLMAP pipeline for locked demo scene.
2. DROID-SLAM if setup is stable and outputs usable poses.
3. ARKit/phone pose metadata only if recording through a controlled app.
4. Other visual odometry/SfM tools only if faster to integrate.

Why this order

The hackathon must not die at pose estimation.

The research story can still include DROID-SLAM as the intended online path, but the judged demo should not depend on live SLAM unless it is proven stable.

Final language

> "DreamNav uses SLAM/SfM pose estimation to recover camera trajectory. For locked demo scenes we use the most reliable reconstruction backend available. The intended online version uses DROID-SLAM-style tracking for faster capture-to-navigation turnaround."

This is honest and strong.

---

16. Inference optimization plan

Custom CUTLASS kernels and FlowRT-style persistent kernel work are not critical-path hackathon features. But basic inference optimization should exist.

This gives DreamNav a concrete ML systems/NVIDIA angle without derailing the build.

Required benchmark paths

Benchmark the completion model under:

1. PyTorch eager FP32
2. PyTorch FP16 / autocast
3. `torch.compile` if stable
4. ONNX Runtime or TensorRT if export is straightforward
5. Cached model-output serving path

Metrics

Report:

- P50 latency
- P95 latency
- throughput if relevant

- VRAM usage if easy
- output resolution
- batch size

Example benchmark table

Runtime path	P50 latency	P95 latency	Notes
PyTorch eager FP32	TBD	TBD	baseline
PyTorch FP16	TBD	TBD	simple optimization
torch.compile	TBD	TBD	if stable
ONNX/TensorRT	TBD	TBD	optional
Cached output	TBD	TBD	demo serving path

NVIDIA-facing sentence

> "For the hackathon I did not write custom kernels, but I profiled the scene completion model and benchmarked practical inference paths — FP16, `torch.compile`, and optional TensorRT — to quantify the latency tradeoff for interactive neural rendering."

This is enough for a strong systems story.

---

17. Product feature priorities

The product features should not distract from the technical core.

Must-have product feature

Lens/FOV preview

This is the most important product feature because it directly maps to production design.

Lens modes:

- 24mm wide
- 35mm standard
- 50mm natural
- 85mm tight/portrait

User value:

- directors can test sightlines
- production designers can understand spatial constraints
- the demo becomes more than a 3D viewer

Should-have product feature

Camera marker/bookmark

Allow the user to save one or more camera positions.

Example labels:

- Wide shot
- Hero angle
- Tracking start
- Close-up

This is useful, but it should not delay the completion model.

Nice-to-have product feature

Scene notes

Scene notes are useful but not technically impressive.

Cut them if time is tight.

Product feature priority

1. Lens/FOV preview
2. Camera marker
3. Scene notes
4. Share link
5. Exported flythrough

---

18. UI screens

Screen 1 — Upload / Demo selection

Purpose:

- select a prevalidated demo scene
- optionally upload a new video

Elements:

- drag/drop upload area
- demo scene thumbnails
- capture guidelines
- quality warning

Recommended copy:

> “Works best with slow, steady indoor walkthroughs with good lighting and textured surfaces.”

Screen 2 — Processing

Purpose:

- teach judges what the system is doing
- make waiting feel meaningful

Stages:

1. Checking capture quality
2. Estimating camera motion
3. Building Gaussian scene
4. Computing visibility support
5. Rendering training views
6. Training geometrically consistent scene model
7. Evaluating held-out viewpoints
8. Applying quality gate
9. Preparing explorer

This screen should explicitly show that a model is being trained/adapted for the specific scene.

Example UI text:

> “Training scene-specific completion model with geometric consistency across nearby viewpoints.”

Screen 3 — Explorer

Purpose:

- main demo

Elements:

- full-screen WebGL scene
- WASD + mouse look
- confidence overlay toggle
- minimap
- lens/FOV selector
- metrics panel
- camera marker button

Screen 4 — Quality report

Purpose:

- technical credibility

Metrics:

- input video length
- frames used
- pose backend
- visibility thresholds
- splat FPS
- number of Gaussians if available
- scene-model training time
- held-out PSNR
- quality gate status
- inference latency by runtime path
- completion cache status
- capture quality score

---

19. API design

`GET /demo-scenes`

Returns available locked demo scenes.

```
[
  {
    "scene_id": "warehouse_01",
    "title": "Warehouse Scout",
    "thumbnail_url": "/thumbs/warehouse_01.jpg",
    "description": "Textured industrial space with partial corner completion"
  }
]
```

`POST /upload`

Accepts video and creates a processing job.

```
{
  "job_id": "scene_abc123",
  "validation_status": "pass",
  "warnings": [],
```

```
  "estimated_processing_time_sec": 240
}
```

`GET /status/{job_id}`

Returns job progress.

```
{
  "job_id": "scene_abc123",
  "stage": "training_scene_model",
  "progress": 0.62,
  "elapsed_sec": 148,
  "message": "Training geometrically consistent scene-specific completion model"
}
```

`WS /progress/{job_id}`

Streams progress updates.

```
{
  "stage": "quality_gate",
  "progress": 0.86,
  "message": "Applying held-out PSNR quality gate"
}
```

`GET /scene/{scene_id}`

Returns scene assets.

```
{
  "scene_id": "warehouse_01",
  "splat_url": "/scenes/warehouse_01/splat.ply",
  "metadata_url": "/scenes/warehouse_01/metadata.json",
  "visibility_manifest_url": "/scenes/warehouse_01/visibility_manifest.json",
  "completion_manifest_url": "/scenes/warehouse_01/completion_manifest.json",
  "quality_report_url": "/scenes/warehouse_01/quality.json"
}
```

`GET /quality/{scene_id}`

Returns metrics.

```
{
  "scene_id": "warehouse_01",
  "pose_backend": "COLMAP",
  "frame_count": 742,
  "visibility_threshold_observed": 3,
  "splat_fps": 42,
  "scene_model_training_sec": 184,
  "heldout_psnr_median": 24.7,
  "quality_gate": "pass",
  "completion_latency_ms_p50": 68,
  "completion_latency_ms_p95": 91,
  "runtime_path": "torch_fp16",
  "cached_completion": true
}
```


---

20. Scene metadata

Each scene should include a metadata file.


```
{
  "scene_id": "apartment_01",
  "title": "Cluttered Apartment",
  "input_video": "apartment_walkthrough.mp4",
  "duration_sec": 32,
  "frame_count": 960,
  "pose_backend": "COLMAP",
  "camera_path": "camera_path.json",
  "splat_file": "splat.ply",
  "visibility": {
    "observed_threshold": 3,
    "partial_threshold": [1, 2],
    "observed_ratio": 0.71,
    "partial_ratio": 0.18,
    "completion_candidate_ratio": 0.11
  },
  "scene_model": {
    "enabled": true,
    "architecture": "pose_conditioned_encoder_decoder",
    "train_views": 520,
    "heldout_views": 80,
    "training_time_sec": 176,
    "loss": "L_rgb + lambda_geo * L_geo",
    "heldout_psnr_median": 24.1,
    "quality_gate": "pass",
```

```
    "lpips": null
  },
  "optimization": {
    "fp32_latency_ms_p50": null,
    "fp16_latency_ms_p50": 68,
    "compiled_latency_ms_p50": null,
    "tensorrt_latency_ms_p50": null,
    "cached_output_latency_ms_p50": 12
  },
  "zones": {
    "observed": "observed_zone.json",
    "partial": "partial_zone.json",
    "completion": "completion_zone.json",
    "unknown": "unknown_zone.json"
  },
  "quality": {
    "capture_score": 0.84,
    "sharpness_score": 0.79,
    "parallax_score": 0.88,
    "texture_score": 0.81,
    "splat_fps": 47,
    "processing_time_sec": 258
  },
  "product_tools": {
    "lens_modes": ["24mm", "35mm", "50mm", "85mm"],
    "camera_markers_enabled": true,
    "notes_enabled": false
  }
}
```


---

21. Instrumentation and evaluation

This section is non-negotiable if DreamNav is meant to help research/career goals.

Every scene should log:

Reconstruction metrics

- input video duration
- frame count
- pose backend
- reconstruction time
- number of Gaussians if available
- splat FPS
- qualitative reconstruction notes

Visibility / confidence metrics

- observed visibility threshold
- partially observed threshold
- observed zone ratio
- partial zone ratio
- completion candidate ratio
- invalid/unknown ratio
- visibility computation time

Scene model metrics

- train view count
- held-out view count
- model architecture
- parameter count if easy
- training time
- training loss curve
- RGB loss curve
- geometric consistency loss curve
- held-out PSNR median
- LPIPS if feasible
- inference latency

Quality gate metrics

- median held-out PSNR
- quality gate status: pass / warning / fail
- whether completion overlay was enabled
- whether completion was shown only with warning

Inference optimization metrics

- PyTorch eager FP32 P50/P95 latency
- FP16 P50/P95 latency
- `torch.compile` P50/P95 latency if stable
- ONNX/TensorRT P50/P95 latency if feasible
- cached output P50/P95 latency

Failure cases

For each failed candidate scene, log:

- reason for failure
- blur/parallax/texture issue
- pose issue
- splat issue
- visibility boundary issue

- model training issue
- geometric consistency issue
- completion issue

This is what turns a hackathon demo into a research asset.

---

22. Baselines

DreamNav needs simple baselines.

Baseline 1 — Splat only

Observed region only. No scene-specific completion.

Question answered:

> What does the scene look like without the completion model?

Baseline 2 — Nearest rendered view

Use nearest splat-rendered view for target pose.

Question answered:

> Does the model improve over simple nearest-view lookup?

Baseline 3 — RGB-only model

Train the same model with only RGB loss.

Question answered:

> Does geometric consistency improve the result compared with pure image interpolation?

This is the most important research baseline for v4.2.

Baseline 4 — Inpainting or simple completion

Optional.

Question answered:

> Is the learned scene-specific model better than a simpler visual fill method?

Required comparison for demo/writeup

At minimum:

Method	What it shows
Raw video	not navigable
3DGS only	observed scene only
RGB-only model	scene-specific prediction without geometry constraint
DreamNav v4.2	observed scene + geometrically consistent scene-specific completion

---

23. Expected visual quality

DreamNav quality depends on whether the region was observed by the input video.

Observed regions

Observed areas can look very detailed if the video is good.

Good expected quality:

- walls
- floors
- ceilings
- large furniture
- couches
- tables
- shelves
- TV stands
- counters
- large lamps

Imperfect expected quality:

- tiny clutter
- thin objects
- transparent objects
- reflective surfaces
- readable text
- object edges
- fast-motion regions

Apartment example

Input scene:

- shelf with books
- TV stand
- speaker
- Xbox
- lamp
- kitchen counter
- food
- knife
- plates
- random clutter

Expected observed-region result:

- shelf shape: good if visible
- books: likely colored blocks/textures, not readable titles
- TV stand: good
- speaker/Xbox: maybe recognizable if large and seen from enough angles
- lamp: recognizable silhouette if not too thin/reflective
- kitchen counter: good
- plates/food/knife: unstable, may smear or blur
- clutter: visually plausible but not semantically reliable

Predicted/completion regions

Predicted regions should not be expected to preserve exact hidden objects.

The model may plausibly infer:

- wall continuation
- floor continuation
- ceiling continuation
- counter continuation
- rough clutter style
- likely furniture-like shapes

The model should not promise:

- exact hidden knife position
- exact book titles
- exact object count
- true geometry behind occlusions
- accurate hidden room layout

Recommended promise:

> **Observed regions are photorealistic when the capture is good. Predicted regions are geometrically constrained and plausible for scouting, not guaranteed factual reconstructions.**

---

24. Quality modes

Quality modes are useful, but they are not critical for the hackathon.

Do not expose developer settings such as:

- Gaussian count
- PSNR threshold
- model checkpoint
- loss weights
- quantization mode

Use product-facing labels.

Future quality modes

Fast Scout

- quicker processing
- lower detail
- rough completion

Standard

- default mode
- best balance
- target hackathon implementation

Cinematic

- longer processing
- better refinement
- higher-quality export

Hackathon decision

Implement **Standard** only.

Mention Fast Scout and Cinematic as roadmap if needed.

Do not create fake clickable modes unless they work.

---

25. Week-by-week build plan

This is the implementation plan for v4.2.

Week 1 — Reconstruction baseline

Goal:

- Get one phone walkthrough into a navigable 3DGS scene.

Tasks:

1. Record 5–8 candidate indoor videos.
2. Choose 2–3 textured scenes.
3. Run reconstruction pipeline.
4. Generate splat for best scene.
5. Load splat in browser viewer.
6. Save camera path metadata.
7. Visualize preliminary visibility counts on the first scene.
8. Calibrate observed/partial/completion thresholds before Week 2 starts.

Calibration rule of thumb:

- If ~70%+ of the scene becomes “completion candidate,” the observed threshold is probably too strict.
- If ~98%+ of the scene becomes “observed,” the threshold is probably too loose.
- Adjust thresholds based on visual inspection before building the overlay UI.

Deliverable:

- one locked navigable demo scene

Cut line:

- if DROID-SLAM is unstable, use COLMAP/existing 3DGS pipeline.

Week 2 — Visibility boundary + overlay foundation

Goal:

- Make the observed/predicted overlay technically real, not arbitrary.

Tasks:

1. Build Next.js/Three.js viewer.
2. Add WASD and mouse look.
3. Add minimap.
4. Show original camera path.
5. Project Gaussians/spatial cells into input cameras.

6. Compute visibility count per Gaussian/cell.
7. Assign observed/partial/completion/unknown zones.
8. Implement overlay toggle using those zones.
9. Store visibility manifest.

Deliverable:

- browser scene with visibility-based observed/predicted overlay

Cut line:

- if per-Gaussian visibility is too hard, use a voxel/spatial-cell approximation first; do not fall back to pure distance-from-path unless clearly labeled as temporary.

Week 3 — Scene-specific completion model with geometric consistency

Goal:

- Build the real ML contribution.

Tasks:

1. Render pseudo-views from the splat.
2. Split train/held-out views.
3. Build small pose-conditioned model.
4. Implement RGB loss.
5. Implement geometric consistency loss.
6. Train model on one scene.
7. Log RGB loss and geometric consistency loss.
8. Evaluate held-out PSNR.
9. Compare against nearest-view baseline.
10. Train or test RGB-only baseline if time permits.

Deliverable:

- geometrically consistent scene-specific model trained and evaluated on one scene

Debug protocol if output is blurry:

1. Verify target pose is actually passed into the model.
2. Normalize pose encoding.
3. Check that pose changes across samples.
4. Confirm output changes when pose changes.
5. Overfit 5–10 views.
6. Verify pose/view pairing and camera intrinsics.
7. Check coordinate-system convention.
8. Switch from L2 to L1 if using L2.
9. Add nearest reference views if pose-only conditioning is too weak.

Cut line:

- do not accept blurry output as “good enough” before debugging pose conditioning and overfitting a tiny subset.
- if still weak after debugging, use the model for low-res predicted regions and keep fallback visuals only for demo safety.

Week 4 — Completion integration + quality gate + inference optimization

Goal:

- Make model output usable inside the viewer and safe to demo.

Tasks:

1. Define completion zone from visibility manifest.
2. Blend model prediction into completion region.
3. Cache model outputs along demo path.
4. Apply PSNR quality gate.
5. Calibrate PSNR thresholds visually on the first demo scene.
6. Check what 20 dB, 22 dB, and higher-PSNR outputs actually look like in the viewer.
7. Adjust the warning/pass gate if visual quality and metric disagree.
8. Disable/warn on completion if gate fails.
9. Add quality panel.
10. Benchmark FP32, FP16, `torch.compile`, optional ONNX/TensorRT.
11. Add completion latency/cache status.
12. Add side-by-side baseline comparison screen or card.

Deliverable:

- observed 3DGS + quality-gated model-predicted completion integrated in the explorer

Cut line:

- if live inference is slow, use cached model outputs but show real benchmark numbers.
- if PSNR < 20 dB, do not show completion as seamless.

Week 5 — Product workflow

Goal:

- Make it feel like a real location scouting tool.

Tasks:

1. Add upload/demo scene selection screen.
2. Add processing timeline.
3. Add lens/FOV preview.
4. Add one camera marker/bookmark feature.

5. Improve visual polish.
6. Add metrics/quality report.

Deliverable:

- complete product demo flow

Cut line:

- cut scene notes and share links if time is tight.

Week 6 — Polish, metrics, rehearsal

Goal:

- Make the demo win.

Tasks:

1. Process 2–3 total demo scenes if possible.
2. Lock the main demo scene.
3. Measure final metrics.
4. Prepare failure-case notes.
5. Record backup demo video.
6. Rehearse exact script.
7. Test on the actual presentation laptop.
8. Prepare short technical appendix.

Deliverable:

- hackathon-ready DreamNav demo with measured technical claims

Cut line:

- no door feature unless everything else is already stable.

---

26. Demo safety strategy

Never rely on an arbitrary live upload during judging.

Use:

- prevalidated input video
- cached scene assets
- cached model predictions along demo path
- local backup
- recorded demo video backup

The live system should look real because it is real, but the judged path should not depend on the weakest failure mode.

Safe demo flow

1. Show original phone video.
2. Select the prevalidated scene.
3. Show processing UI.
4. Open the 3DGS explorer.
5. Walk through observed region.
6. Toggle observed/predicted overlay.
7. Walk into completion zone.
8. Show scene-model prediction.
9. Show lens/FOV preview.
10. Show metrics panel.
11. End with product/research framing.

---

27. Exact demo script

Beat 1 — Setup

> “A location scout in Detroit films a warehouse for 30 seconds. The director in LA does not just need another video — they need to walk the space, test sightlines, and decide whether this location is worth the budget.”
>
> “DreamNav turns that phone walkthrough into an interactive scouting scene.”

Beat 2 — Input video

> “This is the original capture. It is just a normal phone walkthrough. No Matterport rig, no LiDAR requirement, no manual modeling.”

Beat 3 — Processing

> “DreamNav estimates the camera motion, builds a Gaussian reconstruction of what the camera actually saw, then computes which surfaces were supported by multiple camera views.”
>
> “It renders training views from this scene and trains a lightweight scene-specific completion model with a geometric consistency loss, so nearby predicted views agree with each other in 3D.”

Beat 4 — Reconstruction

> “Now we are inside the video. This region is the observed reconstruction, rendered as a 3D Gaussian splat directly in the browser.”

Beat 5 — Overlay

> “The key feature is trust. When I turn on the overlay, the tint is not arbitrary. It comes from visibility support — whether that part of the scene was directly seen by enough input cameras.”
>
> “So the user is never forced to guess what is real and what was inferred.”

Beat 6 — Completion

> “This predicted region comes from a model trained on this specific scene, not a generic text-to-3D prior. It learns from the captured space and predicts nearby unseen viewpoints while being constrained to stay consistent across views.”

Beat 7 — Product feature

> “For production design, I can switch lens previews and check whether the space works for a 24mm wide shot, a 35mm, or a tighter 50mm frame.”

Beat 8 — Metrics

> “We also show the technical report: reconstruction backend, visibility threshold, splat FPS, scene-model training time, held-out PSNR, quality gate status, and inference latency.”

Beat 9 — Close

> “DreamNav is a scene-level world model for partially observed environments. It reconstructs what was seen, predicts nearby unseen viewpoints, and exposes confidence instead of hiding uncertainty.”
>
> “For filmmakers, that means remote scouting. For researchers, it is a compact testbed for scene-specific world modeling under partial observability.”

---

28. Metrics to know by heart

Use measured numbers only. Do not invent final values.

Target metrics to collect:

Metric	Why it matters
Input video length	Shows practical capture size
Frame count	Shows data used
Pose backend	Technical credibility
Reconstruction time	Pipeline performance
Visibility threshold	Overlay credibility
Observed/partial/completion zone ratio	Confidence decomposition
Splat FPS	Interactivity
Number of Gaussians	Scene complexity
Train views	ML credibility
Held-out views	Evaluation credibility
Scene model training time	Test-time adaptation cost
Held-out PSNR median	Novel-view quality
Quality gate status	Demo trust
RGB-only baseline PSNR	Shows value of geometry loss
LPIPS if available	Perceptual quality
Completion latency P50/P95	Real-time usability
Runtime optimization path	Systems credibility
Cache status	Honest serving explanation
Capture quality score	Product trust

Recommended benchmark table:

Component	Measured value
Video length	TBD
Frames used	TBD
Pose backend	TBD
Visibility observed threshold	3 input views provisional
Observed zone ratio	TBD
Splat FPS	TBD
Train views	TBD

Component	Measured value
Held-out views	TBD
Scene model training time	TBD
Held-out PSNR median	TBD
Quality gate	TBD
RGB-only baseline PSNR	TBD
Completion latency P50/P95	TBD
Best runtime path	TBD

29. Risk table

Risk	Severity	Why it matters	Mitigation
Pose estimation fails	Critical	Bad poses break splat and model	Prevalidated demos, fallback backend
Splat quality is poor	Critical	Visual demo fails	Textured scenes, locked demos, test early
Visibility boundary is wrong	High	Overlay becomes misleading	Multi-view visibility support, debug visualization
Completion model ignores pose	High	Output becomes blurry average image	Overfit tiny subset, verify pose conditioning
Geometric consistency implementation breaks	Medium	Research claim weakens	Start approximate, log RGB-only baseline
Held-out PSNR is weak	Medium	Research signal weakens	Quality gate, honest baseline comparison
Live inference is slow	Medium	Interaction feels bad	FP16/compile benchmark, cache model outputs
Browser FPS drops	High	Demo feels broken	Prune scene, cap complexity, test laptop
Product features distract	Medium	Build time wasted	Lens/FOV only as must-have
Door feature fails	Medium	Embarrassing if shown	Stretch only
Arbitrary uploads fail	High	Live demo risk	Do not rely on arbitrary upload during judging

Risk	Severity	Why it matters	Mitigation
Overclaiming hurts credibility	High	Judges/researchers lose trust	Use precise language

30. Objective category ratings for v4.2

Category	Rating	Explanation
Hackathon win potential	8.5/10	Strong visual demo with safe fallback paths
Product signal	8.5/10	AI location scouting is specific and understandable
Physical AI signal	8/10	Strong representation layer, but no action/control
World model signal	9/10	Scene-specific prediction + geometric consistency + partial observability
3D vision signal	9/10	SLAM/SfM + 3DGS + visibility support + browser navigation
ML signal	8.5/10	Stronger due to geometric consistency and held-out evaluation
ML systems signal	8/10	Latency benchmark, FP16/compile/TensorRT path, caching
Research/PI impressiveness	8.5/10	Credible if geometric consistency and baselines are shown
NVIDIA relevance	8.5/10	Neural rendering, latency, GPU pipeline, inference optimization
Recruiter impressiveness	9/10	Easy to explain and visually memorable
Hiring manager impressiveness	8.5/10	Strong end-to-end engineering project
Feasibility	6.8/10	Harder than v4.1, but sharper and more valuable
Technical cohesiveness	9/10	Clear chosen ML backbone and overlay definition
Product cohesiveness	9/10	Clear scouting workflow
Resume value	9/10	Excellent if demo and metrics are real

31. Post-hackathon research path

If the hackathon demo works, extend DreamNav into a research project.

Research question

> How can a system build a confidence-aware, geometrically consistent scene-specific world model from a short video that supports novel-view prediction under partial observability?

Experiments

1. Scene-specific model versus splat-only baseline.
2. Scene-specific model versus nearest-view baseline.
3. Geometric consistency model versus RGB-only model.
4. Scene-specific model versus inpainting baseline.
5. Training steps versus held-out PSNR.
6. Completion quality versus number of rendered pseudo-views.
7. Visibility threshold sensitivity.
8. Confidence overlay calibration.
9. Optional DROID-SLAM online path comparison.
10. Optional perceptual loss ablation.

Possible paper contribution

> **Geometrically consistent scene-specific neural completion for partially observed Gaussian-splat environments with explicit visibility-based observed/predicted confidence decomposition.**

Future JEPA extension

After the hackathon, replace or augment RGB prediction with latent prediction.

Possible direction:

- encode target views into latent embeddings
- train model to predict target latent from context views and target pose
- use latent prediction error as uncertainty
- decode or use latent state for planning-oriented world models

This is relevant to embodied AI/world-model research, but should remain future work for now.

Future VLA extension

Use DreamNav scenes as environments for instruction-following navigation.

Example:

> “Move to the kitchen counter and find a wide camera angle facing the window.”

This would require:

- instruction parsing
- scene semantics
- action/path planning
- navigation policy or planner

Not hackathon scope.

---

32. Resume positioning

General bullet

Built DreamNav, an AI location-scouting system that converts short phone walkthrough videos into browser-navigable 3D Gaussian scenes with visibility-aware, scene-specific neural completion for nearby unseen viewpoints.

ML/research bullet

Trained lightweight scene-specific completion models on splat-rendered pseudo-views with geometric consistency loss, evaluating held-out novel views against RGB-only and nearest-view baselines under partial observability.

3D vision bullet

Implemented a video-to-3D pipeline using SLAM/SfM camera-pose estimation, 3D Gaussian Splatting reconstruction, and multi-view visibility support to separate observed, partial, and model-predicted scene regions.

Systems bullet

Deployed an interactive 3D/ML pipeline with Next.js, FastAPI, WebSockets, GPU-backed processing, cached model outputs, and FP16/compiled inference benchmarks for browser-based scene exploration.

Physical AI/world-model bullet

Built a queryable scene-level world model for partially observed real environments, separating observation-backed geometry from geometrically constrained model-predicted regions for confidence-aware spatial exploration.

---

33. Final MVP definition

The MVP is successful if it does the following:

1. Shows an original phone walkthrough.
2. Reconstructs the observed scene as a navigable 3DGS environment.
3. Lets the user walk through the scene in browser.
4. Shows camera trajectory on a minimap.
5. Computes observed/partial/completion zones from multi-view visibility support.
6. Trains or demonstrates a scene-specific completion model.
7. Uses geometric consistency loss by default.
8. Evaluates the model on held-out views.
9. Applies a PSNR quality gate.
10. Shows a predicted/completion region only if it passes or is clearly labeled low-confidence.
11. Toggles observed versus predicted overlay.
12. Includes lens/FOV preview.
13. Displays real metrics and inference latency.
14. Tells a clear location-scouting story.

The project fails if it becomes only:

> “A 3DGS viewer with cached panels and AI branding.”

The project also fails if it becomes:

> “A giant research system with five fragile modules and no reliable demo.”

The correct middle is:

> **Reliable demo, real ML backbone, visibility-based confidence, geometric consistency, honest uncertainty.**

---

34. Final recommendation

DreamNav v4.2 should be built around one sentence:

> **DreamNav reconstructs what the camera saw with 3D Gaussian Splatting, computes what was truly observed using multi-view visibility, trains a geometrically consistent scene-specific model to predict nearby unseen viewpoints, and shows users exactly what is observed versus inferred.**

That is the project.

Everything else should serve that sentence.

Keep:

- 3DGS
- SLAM/SfM
- visibility-based observed/completion boundary
- scene-specific ML model
- geometric consistency loss
- PSNR quality gate
- confidence overlay
- held-out evaluation
- basic inference optimization
- lens/FOV preview
- product story
- demo safety

Cut from the critical path:

- JEPA
- VLA
- door/corridor generation
- custom CUTLASS kernels
- arbitrary upload promise
- centimeter-accuracy claims
- too many product side-features

This version is strong enough for a hackathon, credible enough for research conversations, and coherent enough for a serious resume project.